

# Bàn về cách giải bài toán trò chơi bất đối xứng

Fang Zhanpeng  
Trường Trung học số 1 Trung Sơn, Quảng Đông

2009

*Nguồn gốc: Bàn về cách giải bài toán trò chơi bất đối xứng*

IOI China candidate team papers / 2009 / Fang Zhanpeng

## Abstract

Bài viết giới thiệu cách xử lý một lớp trò chơi trong đó hai người chơi không có cùng tập nước đi hợp lệ. Nội dung gồm bốn phần: mở đầu bằng sự khác nhau giữa các bài toán trò chơi; dùng surreal number để giải một lớp trò chơi tổ hợp bất đối xứng; quay lại phân tích trạng thái, truy hồi và lặp để xử lý các bài toán không phù hợp với công cụ đó; cuối cùng là tổng kết.

**Từ khóa.** Trò chơi bất đối xứng; surreal number; phân tích trạng thái; truy hồi; lặp.

## 1 Mở đầu

Trong thi Tin học, bài toán trò chơi xuất hiện khá thường xuyên. Xét ví dụ kinh điển *Green Hackenbush*: cho  $n$  cây tre có chiều cao lần lượt là  $a_1, a_2, \dots, a_n$ . Hai người chơi  $L$  và  $R$  chơi trò chặt tre theo các luật sau:

1. Hai người lần lượt thao tác,  $L$  đi trước.
2. Mỗi lượt chọn một cây tre có chiều cao khác 0, chặt bỏ một đoạn của cây đó; mọi phần không còn nối với gốc cũng bị bỏ đi.
3. Người đầu tiên không thể thao tác sẽ thua.

Nếu cả hai đều tối ưu, cần xác định ai thắng ở trạng thái đã cho.

Một cây tre cao  $n$  sau một lần chặt có thể trở thành một cây cao từ 0 đến  $n - 1$ . Vì vậy nó tương đương một đồng Nim có  $n$  viên. Bài toán trở thành Nim với  $n$  đồng có kích thước  $a_1, a_2, \dots, a_n$ : mỗi lượt chọn một đồng và lấy đi ít nhất một viên, người không đi được thua. Theo định lý Sprague–Grundy, nếu  $g_i$  là hàm SG của trò chơi con  $G_i$ , thì trò chơi tổng  $G = G_1 + G_2 + \dots + G_n$  có hàm SG

$$g(x_1, x_2, \dots, x_n) = g_1(x_1) \text{ xor } g_2(x_2) \text{ xor } \dots \text{ xor } g_n(x_n).$$

Do đó bài toán trên có thuật toán  $\mathcal{O}(n)$ .

Trong *Green Hackenbush*, ở mọi trạng thái,  $L$  và  $R$  có cùng tập lựa chọn: nếu  $L$  có thể chặt đoạn thứ  $j$  của cây thứ  $i$ , thì  $R$  cũng có thể. Nếu điều này không còn đúng thì sao? Ta thêm một luật: mỗi đoạn tre được gán nhãn  $L$  hoặc  $R$ ; người chơi  $L$  chỉ được chặt đoạn mang nhãn  $L$ , và  $R$  chỉ được chặt đoạn mang nhãn  $R$ . Khi ấy tập quyết định của hai bên khác nhau, đồng thời định lý Sprague–Grundy không còn áp dụng trực tiếp.

Bài viết này bàn về cách xử lý những trò chơi trong đó hai người chơi có tập nước đi hợp lệ khác nhau; ta gọi chúng là trò chơi bất đối xứng (*partizan games*).

## 2 Một hướng khác

Phần này giới thiệu công cụ *surreal number*, rồi dùng một ví dụ để phân tích cách giải một lớp trò chơi tổ hợp bất đối xứng.

### 2.1 Giới thiệu surreal number

Trong toán học, surreal number tạo thành một trường chứa cả số thực, số vô cùng lớn và số vô cùng bé. Ở đây ta chỉ dùng nó như công cụ phân tích trò chơi, nên các tính chất ít liên quan sẽ không được trình bày.

#### 2.1.1 Cách xây dựng

**Định nghĩa 1.** Một surreal number gồm hai tập, gọi là tập trái và tập phải. Ta thường viết nó dưới dạng  $\{L|R\}$ , trong đó  $L$  là tập trái và  $R$  là tập phải. Các phần tử trong hai tập cũng là surreal number, đồng thời không tồn tại  $x \in R$  và  $y \in L$  sao cho  $x \leq y$ .

**Định nghĩa 2.** Với hai surreal number

$$x = \{X_L|X_R\}, \quad y = \{Y_L|Y_R\},$$

ta nói  $x \leq y$  khi và chỉ khi không tồn tại  $x_L \in X_L$  sao cho  $y \leq x_L$ , và cũng không tồn tại  $y_R \in Y_R$  sao cho  $y_R \leq x$ . Để tiện trình bày, viết  $x \geq y$  thay cho  $y \leq x$ , viết  $x < y$  thay cho  $x \leq y \wedge \neg(y \leq x)$ , viết  $x > y$  thay cho  $y < x$ , và viết  $x = y$  thay cho  $x \leq y \wedge y \leq x$ .

Hai định nghĩa trên đều mang tính đệ quy. Ban đầu ta chưa biết surreal number nào, nên chỉ có thể lấy cả hai tập rỗng, thu được  $\{\emptyset|\emptyset\}$ , thường viết gọn là  $\{|\}$ . Theo định nghĩa 1, đây là một surreal number hợp lệ; đặt  $\{|\} = 0$ , và nói 0 sinh ở ngày thứ 0.

Từ 0, ta thử tạo ra các số mới:

$$\{0|\}, \quad \{|\ 0\}, \quad \{0|0\}.$$

Vì  $0 \leq 0$ ,  $\{0|0\}$  không hợp lệ. Theo định nghĩa 2,

$$\{|\ 0\} < 0 < \{0|\},$$

nên đặt  $\{|\ 0\} = -1$ ,  $\{0|\} = 1$ . Hai số 1 và  $-1$  sinh ở ngày thứ 1.

Dùng 0, 1,  $-1$  làm phần tử của tập trái và tập phải, ta dựng được 17 surreal number hợp lệ ở ngày thứ 2. Chẳng hạn

$$\{1|\} = 2, \quad \{|\ -1\} = -2.$$

Ngoài ra  $0 < \{0|1\} < 1$ . Vì phép cộng sẽ cho  $\{0|1\} + \{0|1\} = 1$ , ta đặt  $\{0|1\} = 1/2$ ; tương tự  $\{-1|0\} = -1/2$ . Như vậy ta đã có

$$-2, -1, -\frac{1}{2}, 0, \frac{1}{2}, 1, 2.$$

Những số hợp lệ còn lại ở ngày thứ 2 có giá trị trùng với một trong các số trên; ví dụ  $\{-1|1\} = 0$ .

Tiếp tục như vậy, ngày thứ 3 cho

$$\frac{1}{4} = \{0 | 1/2\}, \quad \frac{3}{4} = \{1/2 | 1\}, \quad \frac{3}{2} = \{1 | 2\}, \quad 3 = \{2 | \},$$

cùng các số âm tương ứng. Lặp mãi, ta xây dựng được mọi hữu tỉ dạng  $j/2^k$ . Có thể mô tả một phần ánh xạ từ các hữu tỉ này sang surreal number bằng hàm dyadic  $\delta$ :

$$\delta(x) = \begin{cases} \{ | \}, & x = 0, \\ \{ \delta(x-1) | \}, & x > 0, x \in \mathbb{Z}, \\ \{ | \delta(x+1) \}, & x < 0, x \in \mathbb{Z}, \\ \left\{ \delta\left(\frac{j-1}{2^k}\right) \middle| \delta\left(\frac{j+1}{2^k}\right) \right\}, & x = \frac{j}{2^k}, j, k \in \mathbb{Z}, k > 0. \end{cases}$$

Thực ra surreal number còn dựng được toàn bộ số thực, số vô cùng lớn và số vô cùng bé, nhưng phần đó nằm ngoài phạm vi bài viết.

### 2.1.2 Một số định lý cơ bản

**Định lý 1.** Với surreal number  $x = \{L | R\}$ ,  $x$  lớn hơn mọi phần tử của  $L$  và nhỏ hơn mọi phần tử của  $R$ .

**Định lý 2.** Với  $x = \{L | R\}$ , nếu  $L$  có phần tử lớn nhất  $l_{\max}$  thì  $\{l_{\max} | R\} = x$ . Tương tự, nếu  $R$  có phần tử nhỏ nhất  $r_{\min}$  thì  $\{L | r_{\min}\} = x$ . Ví dụ:

$$\{1, 2, 3 | 4, 5\} = \{3 | 4, 5\} = \{1, 2, 3 | 4\} = \{3 | 4\}.$$

Ta không thể sửa định nghĩa surreal number thành “mỗi tập trái và phải có nhiều nhất một phần tử”, vì khi các tập là vô hạn, chúng có thể không có phần tử lớn nhất hoặc nhỏ nhất.

**Định lý 3.** Giả sử surreal number sinh sớm nhất trong khoảng từ  $a$  đến  $b$  sinh ở ngày thứ  $i$ . Khi đó trong khoảng ấy chỉ có đúng một surreal number sinh ở ngày thứ  $i$ .

**Định lý 4.** Nếu  $a < x < b$ , và  $x$  là surreal number sinh sớm nhất giữa  $a$  và  $b$ , thì  $\{a | b\} = x$ .

### 2.1.3 Phép toán

Các phép cộng và trừ của surreal number cũng được định nghĩa đệ quy.

**Định nghĩa 3.** Với

$$x = \{X_L | X_R\}, \quad y = \{Y_L | Y_R\},$$

đặt

$$x + y = \{X_L | X_R\} + \{Y_L | Y_R\} = \{X_L + y, x + Y_L | X_R + y, x + Y_R\},$$

trong đó với tập  $X$  và surreal number  $y$ ,

$$X + y = \{x + y : x \in X\}.$$

Điều kiện dừng của đệ quy là  $\emptyset + n = \emptyset$ .

**Định nghĩa 4.** Với  $x = \{X_L | X_R\}$ , số đối của  $x$  là

$$-x = -\{X_L | X_R\} = \{-X_R | -X_L\},$$

trong đó  $-X = \{-x : x \in X\}$ . Điều kiện dừng là

$$-0 = -\{\mid\} = \{\mid\} = 0.$$

**Định nghĩa 5.** Với surreal number  $x, y$ , đặt  $x - y = x + (-y)$ .

Từ các định nghĩa trên suy ra:

1. Nếu  $x = \{X_L | X_R\}$  và  $y = \{Y_L | Y_R\}$ , thì

$$x + y = \{X_L + y, x + Y_L | X_R + y, x + Y_R\}$$

là một surreal number hợp lệ.

2. Phép cộng giao hoán:  $x + y = y + x$ .

3. Phép cộng kết hợp:  $(x + y) + z = x + (y + z)$ .

## 2.2 Surreal number và trò chơi tổ hợp

### 2.2.1 Định nghĩa và biểu diễn trò chơi tổ hợp

Trước khi áp dụng surreal number, ta cố định lớp trò chơi được xét:

- Có hai người chơi, gọi là  $L$  và  $R$ .
- Ở mọi thời điểm, trò chơi có một trạng thái xác định.
- Một thao tác chuyển trò chơi từ trạng thái hiện tại sang trạng thái khác; luật chơi cho biết từ mỗi trạng thái người chơi có thể tới những trạng thái nào.
- Hai người chơi luân phiên thao tác.
- Nếu tới lượt một người mà người đó không thể thao tác, trò chơi kết thúc. Ở đây chỉ xét quy ước người đầu tiên không thao tác được sẽ thua.
- Với mọi chuỗi thao tác, trò chơi luôn kết thúc sau hữu hạn bước, tức không có hòa.
- Hai người chơi biết đầy đủ luật chơi, trạng thái và mọi thao tác trước đó.

Vì trò chơi được tạo bởi các trạng thái, nếu ở trạng thái  $P$ , tập trạng thái mà  $L$  có thể chuyển tới là  $P_L$ , còn tập trạng thái mà  $R$  có thể chuyển tới là  $P_R$ , ta viết

$$P = \{P_L | P_R\}.$$

Ví dụ, nếu từ  $P$ ,  $L$  có thể tới  $A, B, C$ , còn  $R$  có thể tới  $D$ , thì

$$P = \{A, B, C | D\}.$$

Cách viết này rất giống surreal number. Thật vậy, mọi surreal number đều là một trò chơi, nhưng không phải trò chơi nào cũng là surreal number, vì với trò chơi  $P = \{P_L | P_R\}$  không bắt buộc mọi phần tử của  $P_L$  nhỏ hơn mọi phần tử của  $P_R$ . Sau đây ta chỉ xét các trò chơi có thể biểu diễn bằng surreal number.

### 2.2.2 Áp dụng vào trò chơi tổ hợp

Giả sử trò chơi  $G$  tương đương surreal number  $x$ , và hai người chơi đều tối ưu. Khi đó:

- Nếu  $x > 0$ , người chơi  $L$  thắng dù đi trước hay đi sau.
- Nếu  $x < 0$ , người chơi  $R$  thắng dù đi trước hay đi sau.
- Nếu  $x = 0$ , người đi sau thắng.

Các kết luận này chứng minh được bằng ý tưởng quy nạp. Khi  $x > 0$ , nếu  $L$  đi trước thì vì  $x$  là surreal number,  $L$  luôn có thể chuyển tới một trạng thái  $\geq 0$ . Nếu  $R$  đi trước,  $R$  chỉ có thể chuyển tới một trạng thái  $> 0$ . Vì vậy tập nước đi của  $L$  không bao giờ rỗng trước, và  $L$  thắng. Trường hợp  $x < 0$  đối xứng. Khi  $x = 0$ , nếu  $L$  đi trước thì chỉ có thể chuyển sang trạng thái  $< 0$ , nên  $R$  thắng; nếu  $R$  đi trước thì chỉ có thể chuyển sang trạng thái  $> 0$ , nên  $L$  thắng.

Nhiều trò chơi  $G$  có thể tách thành  $n$  trò chơi con rời nhau  $G_1, G_2, \dots, G_n$ , mỗi thao tác của  $G$  tương đương chọn một trò chơi con rồi thao tác trong đó. Khi ấy ta gọi  $G$  là tổng của các trò chơi con, viết

$$G = G_1 + G_2 + \dots + G_n.$$

Nim nhiều đồng là ví dụ kinh điển.

**Định lý 8.** Nếu trò chơi  $G$  tương đương surreal number  $x$ , và trò chơi  $H$  tương đương surreal number  $y$ , thì trò chơi  $G + H$  tương đương surreal number  $x + y$ .

Định lý này đến trực tiếp từ định nghĩa phép cộng. Nhờ đó, surreal number giải được nhiều trò chơi bất đối xứng mà định lý Sprague–Grundy không xử lý được.

## 2.3 Ví dụ 1: Procrastination

### Tóm tắt đề bài

Trò chơi *Procrastination* có luật như sau:

- Ban đầu có bốn tháp, mỗi tháp là một chồng lập phương đen hoặc trắng.
- Hai người chơi  $L$  và  $R$  luân phiên thao tác.
- Mỗi lượt, người chơi chọn một lập phương rồi bỏ lập phương đó cùng mọi lập phương nằm phía trên nó. Người chơi  $L$  chỉ được chọn lập phương trắng;  $R$  chỉ được chọn lập phương đen.
- Người đầu tiên không thể thao tác sẽ thua.

Một trạng thái ban đầu được gọi là *W-configuration* nếu  $L$  thắng dù đi trước hay đi sau. Một cấu hình con  $C$  gồm ba tháp. Một trạng thái đầy đủ có thể được tạo bởi  $C$  và một tháp thứ tư  $T$ , ký hiệu  $(C, T)$ . Với hai cấu hình con  $C_1, C_2$ , nói  $C_1$  không kém  $C_2$  nếu với mọi tháp  $T$ , hế  $(C_2, T)$  là W-configuration thì  $(C_1, T)$  cũng là W-configuration.

Cho  $C_1, C_2$ , cần kiểm tra  $C_1$  có không kém  $C_2$  hay không. Mỗi tháp đầu vào có chiều cao  $n$ , với  $0 \leq n \leq 50$ .

### Phân tích bằng surreal number

Trước hết xét một trạng thái  $G$  chỉ gồm một tháp  $T$ . Nếu  $T$  rỗng thì cả hai người đều không có nước đi, nên

$$G = \{ \mid \} = 0.$$

Nếu  $T$  chỉ có một lập phương trắng,  $L$  có một nước đi tới 0, nên

$$G = \{0 \mid\} = 1.$$

Nếu  $T$  gồm hai lập phương trắng,  $L$  có thể đưa trò chơi tới 1 hoặc 0, nên

$$G = \{0, 1 \mid\} = 2.$$

Quy nạp cho thấy tháp gồm  $n$  lập phương trắng có giá trị  $n$ ; tháp gồm  $n$  lập phương đen có giá trị  $-n$ .

Tiếp theo xét tháp gồm  $n > 0$  lập phương trắng ở dưới và một lập phương đen ở trên. Khi đó

$$G = \{0, 1, \dots, n-1 \mid n\} = \{n-1 \mid n\} = n - \frac{1}{2}.$$

Nếu thêm một lập phương đen nữa lên trên, ta có

$$G = \left\{ n-1 \mid n - \frac{1}{2} \right\} = n - \frac{1}{2} - \frac{1}{4}.$$

Nếu từ trạng thái đó thêm một lập phương đen ở đỉnh, giá trị thành

$$n - \frac{1}{2} - \frac{1}{4} - \frac{1}{8};$$

còn nếu thêm một lập phương trắng ở đỉnh, giá trị thành

$$n - \frac{1}{2} - \frac{1}{4} + \frac{1}{8}.$$

Với một tháp bất kỳ, giá trị tương ứng được tính bằng thuật toán sau. Ký hiệu  $T[i]$  là màu của lập phương thứ  $i$  tính từ dưới lên.

`SurrealNumber(T)`

```

x <- 0
i <- 1
n <- chiều cao của T
while i <= n and T[i] = T[1]
    if T[i] = trắng then x <- x + 1 else x <- x - 1
    i <- i + 1
k <- 2
while i <= n
    if T[i] = trắng then x <- x + 1 / k else x <- x - 1 / k
    i <- i + 1
    k <- k * 2
return x

```

Thuật toán có hai bước: trước hết xử lý đoạn đáy gồm các lập phương cùng màu, sau đó xử lý từng lập phương phía trên đoạn đó bằng các trọng số  $1/2, 1/4, 1/8, \dots$ . Độ phức tạp là  $\mathcal{O}(n)$ . Các hình trong bản gốc chỉ minh họa các tháp nói trên: một đoạn đáy cùng màu, rồi các lập phương phía trên lần lượt đóng góp các lũy thừa âm của 2 với dấu phụ thuộc màu.

Chứng minh đúng đắn của bước thứ hai dùng quy nạp. Xét tháp  $T$  có  $n$  lập phương dưới cùng cùng màu  $C_1$ , lập phương thứ  $n+1$  có màu khác  $C_1$ , và phía trên đoạn đáy có tổng cộng  $p$  lập phương.

Với  $p = 1$ , giả sử đoạn  $T[1..n]$  có giá trị  $v$ . Nếu lập phương trên cùng là đen, thì

$$G = \{0, 1, \dots, n-1 \mid n\} = \{n-1 \mid n\} = n - \frac{1}{2} = v - \frac{1}{2}.$$

Nếu lập phương trên cùng là trắng, lập luận đối xứng cho

$$G = -n + \frac{1}{2} = v + \frac{1}{2}.$$

Giả sử thuật toán đúng với mọi  $p = 1, \dots, m-1$ , ta chứng minh cho  $p = m$ . Đánh số  $m+1$  lập phương trên cùng từ trên xuống dưới là  $m, m-1, \dots, 0$ .

Nếu lập phương số  $m$  là đen, tìm từ trên xuống lập phương trắng đầu tiên và gọi số của nó là  $k$ . Gọi  $v$  là giá trị của  $T[1..n+k]$ , và  $u$  là giá trị của  $T[1..n+m-1]$ . Theo định lý 2,

$$\begin{aligned} G &= \left\{ v - \frac{1}{2^k} \left| v - \frac{1}{2^{k+1}} - \frac{1}{2^{k+2}} - \dots - \frac{1}{2^{m-1}} \right. \right\} \\ &= \left\{ v - \frac{1}{2^k} \left| v - \frac{1}{2^k} + \frac{1}{2^{m-1}} \right. \right\} \\ &= \left\{ \frac{v2^m - 2^{m-k}}{2^m} \left| \frac{v2^m - 2^{m-k} + 2}{2^m} \right. \right\}. \end{aligned}$$

Theo hàm dyadic, số sinh sớm nhất giữa hai cận trên là

$$G = \frac{v2^m - 2^{m-k} + 1}{2^m} = v - \frac{1}{2^k} + \frac{1}{2^{m-1}} - \frac{1}{2^m} = u - \frac{1}{2^m}.$$

Nếu lập phương số  $m$  là trắng, tìm lập phương đen đầu tiên từ trên xuống và gọi số của nó là  $k$ . Khi đó

$$\begin{aligned} G &= \left\{ v + \frac{1}{2^{k+1}} + \frac{1}{2^{k+2}} + \dots + \frac{1}{2^{m-1}} \left| v + \frac{1}{2^k} \right. \right\} \\ &= \left\{ v + \frac{1}{2^k} - \frac{1}{2^{m-1}} \left| v + \frac{1}{2^k} \right. \right\} \\ &= \left\{ \frac{v2^m + 2^{m-k} - 2}{2^m} \left| \frac{v2^m + 2^{m-k}}{2^m} \right. \right\}. \end{aligned}$$

Theo hàm dyadic,

$$G = \frac{v2^m + 2^{m-k} - 1}{2^m} = v + \frac{1}{2^k} - \frac{1}{2^{m-1}} + \frac{1}{2^m} = u + \frac{1}{2^m}.$$

Vậy thuật toán đúng với mọi  $p$ .

Bây giờ xét trạng thái ban đầu  $G$  gồm  $n$  tháp  $T_1, T_2, \dots, T_n$ , có giá trị tương ứng  $x_1, x_2, \dots, x_n$ . Theo định lý 8,

$$G = x_1 + x_2 + \dots + x_n.$$

Bài toán yêu cầu kiểm tra liệu với mọi tháp  $H$ , từ  $C_2 + H > 0$  có suy ra  $C_1 + H > 0$  hay không. Điều này tương đương kiểm tra  $C_1 \geq C_2$ . Vì vậy chỉ cần tính tổng giá trị các tháp trong từng cấu hình con rồi so sánh. Độ phức tạp là  $\mathcal{O}(n)$ .

## Tiểu kết

Ví dụ trên cho thấy surreal number cho một lời giải rất sáng sủa đối với một lớp trò chơi tổ hợp bất đối xứng có thể tách thành tổng các trò chơi con. Với *Procrastination*, chương trình theo hướng này rất ngắn. Vì vậy, khi trò chơi có cấu trúc phù hợp, surreal number nên là lựa chọn đầu tiên.

## 3 Quay lại điểm xuất phát

Surreal number có lợi thế lớn cho một lớp trò chơi bất đối xứng đặc biệt, nhất là các trò chơi có thể tách thành tổng. Tuy nhiên phạm vi áp dụng của nó khá hẹp; với những trò chơi tổng quát hơn như các trò chơi cờ thường gặp, công cụ này không giúp nhiều. Khi đó ta quay lại phân tích trạng thái từ đầu, thiết kế trạng thái hợp lý và dùng quy hoạch động hoặc lặp.

### 3.1 Ví dụ 2: Procrastination

#### Tóm tắt đề bài

Đề bài giống ví dụ 1.

#### Phân tích bằng trạng thái

Từ phần trước, ta biết surreal number giải bài này rất dễ. Nhưng nếu không dùng công cụ đó, vẫn có thể phân tích trực tiếp.

Định nghĩa W-configuration nói rằng  $L$  thắng dù đi trước hay đi sau. Vì nó liên quan cả hai khả năng, ta tìm cách đơn giản hóa.

**Mệnh đề 3.1.1.** Với một trạng thái  $G$ , nếu  $L$  thắng khi đi trước, thì  $L$  cũng thắng khi đi sau.

*Chứng minh.* Nếu  $L$  thắng khi đi trước trong  $G$ , tồn tại một lập phương trắng  $K$  mà nếu  $L$  chọn  $K$ , trò chơi chuyển sang trạng thái  $G_1$  trong đó  $R$  đi trước và thua. Khi  $R$  đi trước trong  $G$ , nếu suốt ván  $R$  không động tới lập phương nào phía trên  $K$ , thì  $R$  tương đương đi trước trong  $G_1$ , nên thua. Nếu  $R$  có động tới phần phía trên  $K$ ,  $L$  lập tức chọn  $K$  sau nước đó, và  $R$  vẫn tương đương đi trước trong  $G_1$ . Vậy  $R$  thua.

Nhờ kết luận này, chỉ cần xét liệu  $L$  có thắng khi đi trước hay không. Định nghĩa  $C_1$  không kém  $C_2$  có lượng từ “mọi tháp  $T$ ”. Dù số tháp là vô hạn, ta thử dùng liệt kê để tìm cấu trúc.

Gọi  $g(X) = 1$  nếu  $L$  thắng khi đi trước trong trạng thái  $X$ , và  $g(X) = 0$  nếu không. Gọi  $\text{add}(T_1, T_2)$  là tháp tạo bởi việc đặt  $T_2$  lên trên  $T_1$ . Ký hiệu  $T[i]$  là lập phương thứ  $i$  từ dưới lên;  $W$  là trắng,  $B$  là đen. Gọi mệnh đề  $P$  là “ $C_1$  không kém  $C_2$ ”.

Cách liệt kê tự nhiên là bắt đầu với tháp rỗng, rồi liên tục đặt thêm một lập phương trắng hoặc đen lên đỉnh; hình trong bản gốc chỉ là cây nhị phân sinh các tháp theo cách đó. Giả sử đang xét một tháp  $T$  với

$$g((C_1, T)) = g((C_2, T)).$$

Ta xét việc đặt thêm lập phương lên  $T$ .

**Mệnh đề 3.1.2.** Nếu  $g((C, T)) = 1$ , thì với mọi tháp  $T_1$  có  $T_1[1] = W$ ,

$$g((C, \text{add}(T, T_1))) = 1.$$

*Chứng minh.* Đặt  $T_2 = \text{add}(T, T_1)$ , chiều cao của  $T$  là  $n$ , chiều cao của  $T_2$  là  $m$ . Nếu trong cả ván  $R$  không thao tác lên lập phương nào phía trên  $T_2[n]$ , thì  $(C, T_2)$  tương đương  $(C, T)$ , nên  $L$  thắng khi đi trước. Nếu  $R$  thao tác phía trên  $T_2[n]$ ,  $L$  lập tức chọn  $T_2[n+1]$ . Trạng thái ban đầu lại tương đương  $(C, T)$ , nên  $L$  vẫn thắng.

Tương tự:

**Mệnh đề 3.1.3.** Nếu  $g((C, T)) = 0$ , thì với mọi tháp  $T_1$  có  $T_1[1] = B$ ,

$$g((C, \text{add}(T, T_1))) = 0.$$

Nếu ở tháp hiện tại  $T$  ta có

$$g((C_1, T)) = g((C_2, T)) = 1,$$

thì nhờ mệnh đề 3.1.2, sau đó chỉ cần liệt kê nhánh đặt thêm một lập phương đen. Nếu

$$g((C_1, T)) = g((C_2, T)) = 0,$$

thì nhờ mệnh đề 3.1.3, chỉ cần liệt kê nhánh đặt thêm một lập phương trắng. Nhờ vậy lượng liệt kê giảm từ bậc lũy thừa xuống bậc tuyến tính theo chiều cao cần xét.



**Mệnh đề 3.1.4.** Nếu  $g((C_1, T)) = 1$  và  $g((C_2, T)) = 0$ , thì với mọi tháp  $T_1$  thỏa

$$g((C_2, \text{add}(T, T_1))) = 1,$$

ta cũng có

$$g((C_1, \text{add}(T, T_1))) = 1.$$

*Chứng minh.* Vì  $g((C_2, T)) = 0$ , nếu  $g((C_2, \text{add}(T, T_1))) = 1$ , mệnh đề 3.1.3 cho  $T_1[1] = W$ . Lại có  $g((C_1, T)) = 1$ , nên mệnh đề 3.1.2 cho kết luận.

Do đó, nếu trong quá trình liệt kê gặp  $T$  với

$$g((C_1, T)) = 1, \quad g((C_2, T)) = 0,$$

thì với mọi tháp khiến  $(C_2, T)$  là W-configuration,  $(C_1, T)$  cũng là W-configuration; mệnh đề  $P$  đúng. Ngược lại, nếu gặp

$$g((C_1, T)) = 0, \quad g((C_2, T)) = 1,$$

thì  $P$  sai.

Còn phải biết khi nào dừng liệt kê, vì có thể tồn tại  $C_1, C_2$  sao cho với mọi  $T$ ,

$$g((C_1, T)) = g((C_2, T)).$$

Trường hợp  $C_1 = C_2$  là ví dụ đơn giản; hình minh họa trong bản gốc cho thêm một ví dụ tương tự.

**Mệnh đề 3.1.5.** Giả sử cấu hình con  $C$  gồm ba tháp có chiều cao  $n_1, n_2, n_3$ . Khi đó  $g((C, T))$  chỉ phụ thuộc vào  $n_1 + n_2 + n_3 + 1$  lập phương thấp nhất của  $T$ .

Vì mỗi tháp đầu vào có chiều cao không quá 50, nếu đã liệt kê tới tháp  $T$  cao hơn 151 mà vẫn có

$$g((C_1, T)) = g((C_2, T)),$$

thì với mọi tháp  $T$  hai giá trị đều bằng nhau, và  $P$  đúng.

Tiếp theo là cách hiện thực hàm  $g$ . Với trạng thái gồm bốn tháp có chiều cao  $n_1, n_2, n_3, n_4$ , định nghĩa

$$d[i, j, k, l, \text{cur}]$$

là trạng thái thắng hay thua khi bốn tháp đã bị giảm xuống các chiều cao  $i, j, k, l$ , và tới lượt người chơi  $\text{cur}$ . Có thể dùng quy hoạch động từ dưới lên; với mỗi trạng thái  $S$ , liệt kê mọi trạng thái  $S'$  đi được trong một nước, rồi xác định  $S$  thắng hay thua. Cách trực tiếp có số trạng thái  $\mathcal{O}(n^4)$ , mỗi trạng thái tốn  $\mathcal{O}(n)$ , nên tổng là  $\mathcal{O}(n^5)$ , quá lớn.

**Mệnh đề 3.1.6.** Trong một trạng thái  $G$ , nếu chọn lập phương  $k$  là một nước thắng cho người đang đi, thì chọn bất kỳ lập phương cùng màu nào nằm phía trên  $k$  cũng là một nước thắng.

*Chứng minh.* Giả sử người đang đi là  $L$ , và chọn lập phương trắng  $k$  là nước thắng. Nếu  $L$  chọn một lập phương trắng  $k'$  phía trên  $k$ , thì sau đó nếu  $R$  không động tới lập phương đen nào phía trên  $k$ , ván chơi tương đương với việc  $L$  chọn  $k$  ngay từ đầu, nên  $L$  thắng. Nếu  $R$  có động tới phía trên  $k$ ,  $L$  lập tức chọn  $k$ , và ván vẫn tương đương trường hợp ban đầu. Do đó chọn  $k'$  cũng thắng. Trường hợp của  $R$  đối xứng.

Vì vậy khi tính một trạng thái, ta không cần thử mọi quyết định; trên mỗi tháp chỉ cần xét lập phương trắng hoặc đen cao nhất phù hợp. Chi phí mỗi trạng thái giảm xuống  $\mathcal{O}(1)$ , nên một trạng thái đầy đủ tốn  $\mathcal{O}(n^4)$ .

Trong quá trình liệt kê tháp mới, tháp mới chỉ khác tháp cũ bởi một lập phương mới đặt thêm, nên không cần tính lại toàn bộ bảng; chỉ cần tính các trạng thái mới phát sinh. Vì chiều cao tháp được liệt kê cũng là bậc  $n$ , độ phức tạp toàn bộ là  $\mathcal{O}(n^4)$ . Với  $n \leq 50$ , cách này đủ dùng.

## Tiểu kết

Cách phân tích này chỉ dùng các trạng thái đặc biệt và quy hoạch động để tính thắng thua, không cần công cụ quá cao cấp. So với lời giải bằng surreal number ở ví dụ 1, nó phải phân tích nhiều trường hợp hơn và hiện thực kém gọn hơn. Tuy nhiên khi gặp một trò chơi mới mà chưa có công cụ sẵn, quay về phân tích trạng thái cơ bản thường là cách mở đường hiệu quả.

### 3.2 Ví dụ 3: The Easy Chase

#### Tóm tắt đề bài

Hai người chơi một trò cờ trên bàn  $n \times n$ . Người chơi thứ nhất có một quân trắng ban đầu ở  $(w_x, w_y)$ ; người chơi thứ hai có một quân đen ban đầu ở  $(b_x, b_y)$ .  $L$  cầm trắng đi trước,  $R$  cầm đen đi sau.

Khi tới lượt  $L$ , quân trắng chọn một trong bốn hướng lên, xuống, trái, phải và đi đúng một ô. Khi tới lượt  $R$ , quân đen cũng chọn một trong bốn hướng và đi một hoặc hai ô. Không quân nào được đi ra ngoài bàn. Một người thắng khi nước đi của mình đưa quân của mình tới ô hiện tại của quân đối phương.

Cả hai dùng cùng tiêu chí chiến lược: nếu có thể thắng, họ thắng trong số bước ít nhất; nếu chắc thua, họ kéo dài ván trong số bước nhiều nhất. Cần dự đoán người thắng và số bước của ván cờ. Ràng buộc:  $2 \leq n \leq 20$ .

#### Phân tích

Đây là một trò chơi bất đối xứng điển hình; các trò cờ quen thuộc như cờ tướng hay cờ vua có thể xem là phiên bản phức tạp hơn. Vì bàn cờ nhỏ và chỉ có hai quân, ta biểu diễn toàn bộ trạng thái có thể xuất hiện rồi tính kết quả của từng trạng thái.

Một trạng thái được mô tả bằng bộ năm

$$(x_1, y_1, x_2, y_2, cur),$$

trong đó  $(x_1, y_1)$  là vị trí quân trắng,  $(x_2, y_2)$  là vị trí quân đen, và  $cur$  cho biết tới lượt  $L$  hay  $R$ . Đề bài cần cả người thắng lẫn số bước, nhưng nếu lưu hai biến *winner* và *move* thì chuyển trạng thái hơi cồng kềnh. Ta mã hóa bằng một giá trị  $f(G)$ . Chọn *infinite* là một số nguyên dương rất lớn, chẳng hạn  $10^8$ . Nếu trạng thái  $G$  do  $L$  thắng sau  $x$  bước, đặt

$$f(G) = infinite - x.$$

Nếu  $G$  do  $R$  thắng sau  $x$  bước, đặt

$$f(G) = -infinite + x.$$

Ví dụ, nếu  $H$  do  $L$  thắng sau 5 bước thì  $f(H) = infinite - 5$ . Nếu  $f(H) = -infinite + 3$ , thì  $H$  do  $R$  thắng sau 3 bước.

Đặt

$$G_L = (w_x, w_y, b_x, b_y, L).$$

Mục tiêu là tính  $f(G_L)$ . Do quan hệ topo giữa các trạng thái không rõ ràng, ta xác định trước giá trị của các trạng thái kết thúc rồi suy ngược. Có hai dạng trạng thái kết thúc:

$$(x, y, x, y, L), \quad (x, y, x, y, R).$$

Khi hai quân đã ở cùng ô và tới lượt  $L$ , nghĩa là  $R$  vừa bắt được quân trắng, nên

$$f(x, y, x, y, L) = -infinite.$$

Tương tự,

$$f(x, y, x, y, R) = \text{infinite}.$$

Vì  $L$  muốn thắng nhanh nhất nếu thắng và thua muộn nhất nếu thua, mục tiêu của  $L$  là làm  $f(G)$  lớn nhất. Ngược lại, mục tiêu của  $R$  là làm  $f(G)$  nhỏ nhất. Ta có các chuyển trạng thái:

$$\begin{aligned} f(x_1, y_1, x_2, y_2, L) &= \max \{f(x'_1, y'_1, x_2, y_2, R) - \text{sign}(f(x'_1, y'_1, x_2, y_2, R))\}, \\ f(x_1, y_1, x_2, y_2, R) &= \min \{f(x_1, y_1, x'_2, y'_2, L) - \text{sign}(f(x_1, y_1, x'_2, y'_2, L))\}. \end{aligned}$$

Ở đây  $(x_1, y_1) \neq (x_2, y_2)$ ;  $(x'_1, y'_1)$  là vị trí quân trắng có thể tới sau một bước;  $(x'_2, y'_2)$  là vị trí quân đen có thể tới sau một bước hợp lệ của nó; và

$$\text{sign}(x) = \begin{cases} 1, & x > 0, \\ 0, & x = 0, \\ -1, & x < 0. \end{cases}$$

Vấn đề còn lại là thứ tự tính. Vì các trạng thái có thể phụ thuộc lẫn nhau, khó chọn một thứ tự topo rõ ràng. Tuy nhiên giá trị của một trạng thái không thể cuối cùng lại suy ngược về chính nó theo kiểu chu trình âm trong bài toán đường đi ngắn nhất. Vì vậy có thể dùng thuật toán lặp tương tự SPFA:

Count( $G'$ )

```

khoi tao f(G) = 0 cho moi trang thai G
liet ke moi trang thai ket thuc Ge
  xac dinh f(Ge)
  dua Ge vào hàng đợi q
while q không rỗng
  lấy phần tử đầu hàng đợi, gọi là Y
  for mỗi trạng thái X có thể đi một bước tới Y
    tmp <- f(X)
    tính lại f(X) theo công thức chuyển trạng thái
    if tmp != f(X) và X không nằm trong q
      đưa X vào q
return f(G')
```

Số trạng thái là  $\mathcal{O}(n^4)$ , số chuyển cùng bậc với số trạng thái, nên độ phức tạp là  $\mathcal{O}(kn^4)$ , trong đó ở trung bình  $k$  là một hằng số nhỏ. Với  $n \leq 20$ , thuật toán xử lý tốt bài toán.

## Mở rộng

Thuật toán trên không chỉ áp dụng cho bài này. Với nhiều trò chơi có số trạng thái không lớn, nếu quan hệ giữa các trạng thái rõ ràng, ta thường dùng quy hoạch động theo thứ tự topo. Khi thứ tự ấy không rõ hoặc khó xác định, lặp theo kiểu trên là một công cụ mạnh và dễ hiện thực.

## 4 Tổng kết

Muốn giải tốt trò chơi bất đối xứng, cần linh hoạt dùng nhiều cách phân tích và kết hợp các tư tưởng thuật toán sâu hơn để đạt hiệu quả. Khi thi Tin học phát triển, các bài toán trò chơi ngày càng đa dạng. Hy vọng các ý tưởng trong bài viết giúp người đọc có thêm một hướng suy nghĩ khi gặp trò chơi. Để xử lý tốt trong thời gian thi hạn hẹp, vẫn phải suy nghĩ và thực hành nhiều; kiến thức trên giấy chỉ thật sự hữu ích khi đã được tự mình rèn luyện.

**Lời cảm ơn.** Tác giả cảm ơn huấn luyện viên Tang Wenbin và Hu Weidong đã hướng dẫn; cảm ơn thầy Ruan Zhiyuan và Wan Jie vì nhiều năm huấn luyện, hướng dẫn; cảm ơn bạn Huang Ziqi ở Trường Trung học số 1 Trung Sơn đã hỗ trợ.

## Tài liệu tham khảo.

1. J. H. Conway, *On Numbers and Games*.
2. Claus Tondering, *Surreal Numbers: An Introduction*.
3. Thomas S. Ferguson, *Game Theory*.
4. Zhang Yifei, bài viết đội tuyển quốc gia năm 2002 về quá trình giải một lớp trò chơi từ trực giác tới lý luận.
5. Luo Ji, bài viết đội tuyển quốc gia năm 2002 về hai hướng tư duy cho bài toán đối sách, bắt đầu từ trò lấy đá.
6. Wang Xiaoke, bài viết đội tuyển quốc gia năm 2007 về phân tích một lớp trò chơi tổ hợp.

## A Đề gốc

### A.1 Procrastination

Ngày xưa có hai nghiên cứu sinh là bạn thân. Trong những lúc nghỉ ngắn khỏi nghiên cứu, thường không quá vài giờ, họ thích chơi trò *Procrastination*.

Đây là trò chơi hai người, đen và trắng, luân phiên đi. Trò chơi thao tác trên các tháp lập phương; mỗi lập phương có màu đen hoặc trắng. Ban đầu có bốn tháp, mỗi tháp là một chồng gồm vài lập phương. Ở lượt của mình, người chơi có thể bỏ bất kỳ lập phương nào cùng màu với mình: người trắng chỉ bỏ lập phương trắng, người đen chỉ bỏ lập phương đen. Mọi lập phương nằm phía trên lập phương được chọn cũng bị bỏ khỏi tháp, bất kể màu gì. Ví dụ, nếu một tháp từ dưới lên là đen, trắng, đen, trắng, thì người đen chọn lập phương đen dưới cùng sẽ bỏ cả tháp; người đen cũng có thể chọn lập phương thứ ba và bỏ luôn lập phương thứ tư. Người trắng chọn lập phương thứ hai thì chỉ còn lại một lập phương đen; người trắng cũng có thể chọn lập phương thứ tư. Người không thể bỏ lập phương nào sẽ thua.

Hai người đã luyện trò này từ thời đại học và chơi hoàn hảo: nếu một người có chiến lược thắng, người đó sẽ thắng. Một lúc nào đó, họ phát hiện rằng với hầu hết cấu hình ban đầu, một người có chiến lược thắng bất kể ai đi trước. Họ gọi một cấu hình là W-configuration nếu trắng có chiến lược thắng bất kể ai đi trước, và B-configuration nếu đen có chiến lược thắng bất kể ai đi trước.

Họ cũng nhận thấy một số cấu hình con ít nhất thuận lợi cho một người chơi bằng những cấu hình con khác. Một cấu hình con  $C$  gồm ba tháp;  $C$  cùng một tháp thứ tư  $T$  tạo thành một cấu hình đầy đủ, ký hiệu  $(C, T)$ . Một cấu hình con  $C_1$  ít nhất thuận lợi cho trắng bằng cấu hình con  $C_2$  nếu và chỉ nếu với mọi tháp thứ tư  $T$ , hế  $(C_2, T)$  là W-configuration thì  $(C_1, T)$  cũng là W-configuration. Nói cách khác, không tồn tại tháp thứ tư  $T$  sao cho  $(C_1, T)$  không phải W-configuration còn  $(C_2, T)$  là W-configuration.

Cho hai cấu hình con  $C_1$  và  $C_2$ , hãy kiểm tra liệu  $C_1$  có ít nhất thuận lợi cho trắng bằng  $C_2$  hay không.

**Dữ liệu vào.** Dòng đầu chứa số bộ kiểm thử. Mỗi bộ gồm một dòng Test  $N$ , trong đó  $N$  là số thứ tự bộ kiểm thử, tiếp theo là tám dòng mô tả lần lượt hai cấu hình con  $C_1, C_2$ . Mỗi cấu hình con gồm bốn dòng. Dòng đầu chứa ba số  $n_1, n_2, n_3$ , là chiều cao ba tháp, với

$$0 \leq n_1, n_2, n_3 \leq 50.$$

Dòng thứ hai chứa  $n_1$  chữ cái B hoặc W, cách nhau bởi dấu cách, mô tả tháp thứ nhất. Dòng thứ ba và thứ tư mô tả hai tháp còn lại tương tự. Chữ W là lập phương trắng, B là lập phương đen. Mỗi tháp được mô tả từ dưới lên trên.

**Dữ liệu ra.** Với mỗi bộ kiểm thử, in trên một dòng số thứ tự bộ kiểm thử và Yes nếu  $C_1$  ít nhất thuận lợi cho trắng bằng  $C_2$ ; ngược lại in No.

**Ví dụ vào.**

```
2
Test 1
331
WBB
WBW
B
333
BWW
BWW
WBB
Test 2
332
WBB
WBW
BB
333
BWW
BWW
WBB
```

**Ví dụ ra.**

```
Test 1: Yes
Test 2: No
```

## A.2 The Easy Chase

Hai người chơi một trò đơn giản trên bàn  $n \times n$ . Người chơi thứ nhất có một quân trắng ban đầu ở  $(rowWhite, colWhite)$ . Người chơi thứ hai có một quân đen ban đầu ở  $(rowBlack, colBlack)$ . Tọa độ đánh số từ 1. Hai người luân phiên đi, người thứ nhất đi trước.

Khi tới lượt người thứ nhất, người đó chọn một trong bốn hướng lên, xuống, trái, phải và di quân trắng một ô theo hướng ấy. Khi tới lượt người thứ hai, người đó cũng chọn một trong bốn hướng và di quân đen một hoặc hai ô. Một người thắng khi nước đi của người đó đưa quân của mình tới ô đang có quân đối phương.

Cả hai dùng chiến lược tối ưu. Nếu có thể thắng, họ chọn chiến lược làm số nước của ván nhỏ nhất. Nếu không thể thắng, họ chọn chiến lược làm số nước của ván lớn nhất. Nếu người thứ nhất thắng, trả về "WHITE x"; nếu người thứ hai thắng, trả về "BLACK x", trong đó x là số nước của ván.

**Định nghĩa.**

Class: TheEasyChase

Method: winner

Parameters: int, int, int, int, int

Returns: String

Method signature:

```
String winner(int n, int rowWhite, int colWhite,  
              int rowBlack, int colBlack)
```

Phương thức phải là public.

**Ràng buộc.**

$$2 \leq n \leq 20.$$

Các tọa độ đều nằm trong khoảng từ 1 đến  $n$ , và hai quân ban đầu nằm ở hai ô khác nhau.

**Ví dụ.**

0)

$n = 2$

rowWhite = 1

colWhite = 1

rowBlack = 2

colBlack = 2

Returns: "BLACK 2"

Người thứ nhất có hai nước đi khả dĩ, nhưng kiểu gì cũng thua.

1)

$n = 2$

rowWhite = 2

colWhite = 2

rowBlack = 1

colBlack = 2

Returns: "WHITE 1"

Ván này kết thúc chỉ sau một nước.

2)

$n = 3$

rowWhite = 1

colWhite = 1

rowBlack = 3

colBlack = 3

Returns: "BLACK 6"

## **B Chương trình tham khảo**

Trong lớp văn bản trích xuất được từ PDF bằng pdftotext, phần phụ lục chương trình chỉ còn các nhan đề “chương trình ví dụ 1”, “chương trình ví dụ 2” và “chương trình ví dụ 3”, không có thân mã nguồn. Vì vậy bản dịch giữ phần phụ lục ở mức ghi chú này và không dựng lại chương trình từ suy đoán.